

Sequential Composition, Feedback, and Traces

This chapter studies the passage from combinational composition to stateful and cyclic composition. In the previous chapter, modules were connected through interfaces and refinement maps; here we add time, iteration, and feedback. The central question is how sequential composition behaves when outputs are fed back into inputs, and which algebraic structures correctly model that behavior without losing compositionality.

A useful starting point is the distinction between open and closed behavior. An open component exposes ports and can be composed with other components by wiring outputs to inputs. A closed component has no external interface and can be analyzed as a self-contained dynamical object. Sequential composition turns a family of open components into a larger open component, while feedback identifies selected outputs with selected inputs to form a loop. The resulting semantics must account for causality, delay, and the possibility of unstable or metastable behavior.

In hardware terms, feedback is not merely a wiring convenience. When a signal is routed back through a combinational path, the resulting loop may admit multiple fixed points, no fixed point, or transient oscillations. For this reason, the semantics of feedback must be guarded by delay, storage, or an explicit fixed-point principle. A common quantitative summary of this risk is the metastability probability $\varepsilon(r)$, viewed as a function of a relevant operating parameter r . In reliability analysis, one often reports a mean time between failures (MTBF) under feedback, emphasizing that the presence of a loop can change the failure profile even when the underlying combinational blocks are individually reliable.

The simplest disciplined way to introduce feedback is through state. A stateful component can be described as a transition system or as a machine with memory. Mealy machines produce outputs as a function of the current state and current input, while Moore machines produce outputs as a function of the current state alone. Both models support sequential composition: the output of one machine becomes the input of the next, and the internal state evolves step by step. Their difference matters in implementation, because Mealy-style outputs can react immediately to inputs, whereas Moore-style outputs are delayed by one state update. This distinction is often reflected in synthesis, verification, and timing closure.

Guarded recursion provides an abstract principle for defining such stateful and cyclic systems. A recursive specification is guarded when each recursive call is mediated by a constructor, delay, or other causal operator. Guardedness prevents ill-founded instantaneous self-reference and ensures that recursive equations can be interpreted as productive or contractive definitions. In categorical language, guarded recursion is closely related to traced structure and to fixed-point operators that are compatible with composition.

Traced categories supply a general algebraic framework for feedback. In a traced monoidal category, one can feed an output wire back into an input wire while preserving the coherence laws of composition and tensoring. The trace abstracts the operation of closing a loop and is governed by axioms that express naturality, dinaturality, vanishing, superposing, and yanking. These laws ensure that feedback is not an ad hoc operation but a disciplined extension of sequential composition. In circuit semantics, the trace captures the idea that a loop can be moved, decomposed, or eliminated when the surrounding structure permits it.

The traced perspective also clarifies the relationship between circuits and automata. A finite-state automaton can be seen as a sequential machine whose transition structure is compatible with feedback through state. Conversely, many feedback circuits can be abstracted as automata by identifying a finite set of internal states and transition rules. This correspondence is especially useful when one wants to reason about equivalence: two implementations may differ syntactically while inducing the same input-output behavior over time.

A practical design question is how to realize memory elements from simpler gates. Flip-flops and latches can be built from NAND or NOR gates, and these constructions illustrate the boundary between combinational and sequential behavior. A latch is typically level-sensitive, while a flip-flop is edge-triggered or otherwise synchronized to a clock. The synchronous case is easier to analyze compositionally because state updates occur at designated times, which helps enforce a global notion of causality. Asynchronous closure is more delicate: without a shared clock or explicit delay discipline, feedback may create race conditions, hazards, or metastable states. The chapter therefore distinguishes synchronous closure, where composition respects a clocked update discipline, from asynchronous closure, where the semantics must explicitly manage timing and stabilization.

Automata minimization provides another lens on sequential composition. Given a machine, one may quotient its states by behavioral equivalence to obtain a smaller machine with the same observable behavior. This reduction preserves the symmetries that matter for external behavior while eliminating redundant internal distinctions. In the context of this book, minimization is not only an optimization technique but also a closure phenomenon: the class of behaviors represented by a machine is stable under quotienting by an appropriate equivalence relation. When symmetries are preserved, the minimized automaton retains the structural invariants relevant to composition and verification.

The equivalence problem becomes especially important when a design is implemented in hardware description language and then checked against a formal specification. The artifact of interest in this chapter is an HDL model paired with formal equivalence checking, typically mediated by an SMT-based backend. The workflow is straightforward in principle: write the sequential design in HDL, derive or encode the intended specification, and discharge the equivalence obligation with a solver. The solver checks that the implementation and specification agree on all admissible inputs and all reachable states, subject to the chosen abstraction. This artifact-oriented approach aligns with the book’s broader claims-as-builds methodology: a semantic claim about feedback or state is accepted only when it is accompanied by a reproducible verification pipeline.

From a categorical standpoint, the main lesson is that sequential composition and feedback are not separate topics. Sequential composition builds larger systems from smaller ones; feedback closes systems into self-referential dynamics; traces provide the algebraic interface between the two. From a hardware standpoint, the main lesson is that state, timing, and equivalence must be treated together. A design is not fully understood until one knows how it composes, how it closes under feedback, how it behaves under minimization, and how its implementation can be checked against its specification.

The remainder of the chapter develops these ideas in a progression from machine models to traced semantics and then to verification artifacts. The intended outcome is a unified view in which sequential composition, feedback, and trace are different presentations of the same compositional principle, constrained by causality and validated by formal equivalence.

Chapter focus. The source material for this chapter emphasizes four concrete threads: the metastability profile $\varepsilon(r)$ and MTBF under feedback; Mealy and Moore machines together with guarded recursion and traced categories; the construction of flip-flops and latches from NAND and NOR gates, including the distinction between synchronous and asynchronous closure; and automata minimization together with preserved symmetries. The chapter closes with an artifact-oriented workflow in which an HDL implementation is checked against a specification using formal equivalence, typically via SMT.

Reading guide. Readers should interpret the later sections of this chapter as progressively refining the same theme. First, sequential composition is treated as ordinary wiring of stateful components. Next, feedback is formalized as a trace or guarded recursion. Then the hardware realization of memory elements is used to connect the abstract semantics to clocked and unclocked circuits. Finally, equivalence checking turns the semantic account into a reproducible verification artifact.

Outlook. These constructions prepare the transition to the next chapter, where constructive logic provides a different but compatible language for digital composition. There, the emphasis shifts from state and feedback to logical structure, but the same compositional discipline remains in force.